

¿Cómo sabes que tu RAG funciona? Métricas de evaluación desde cero



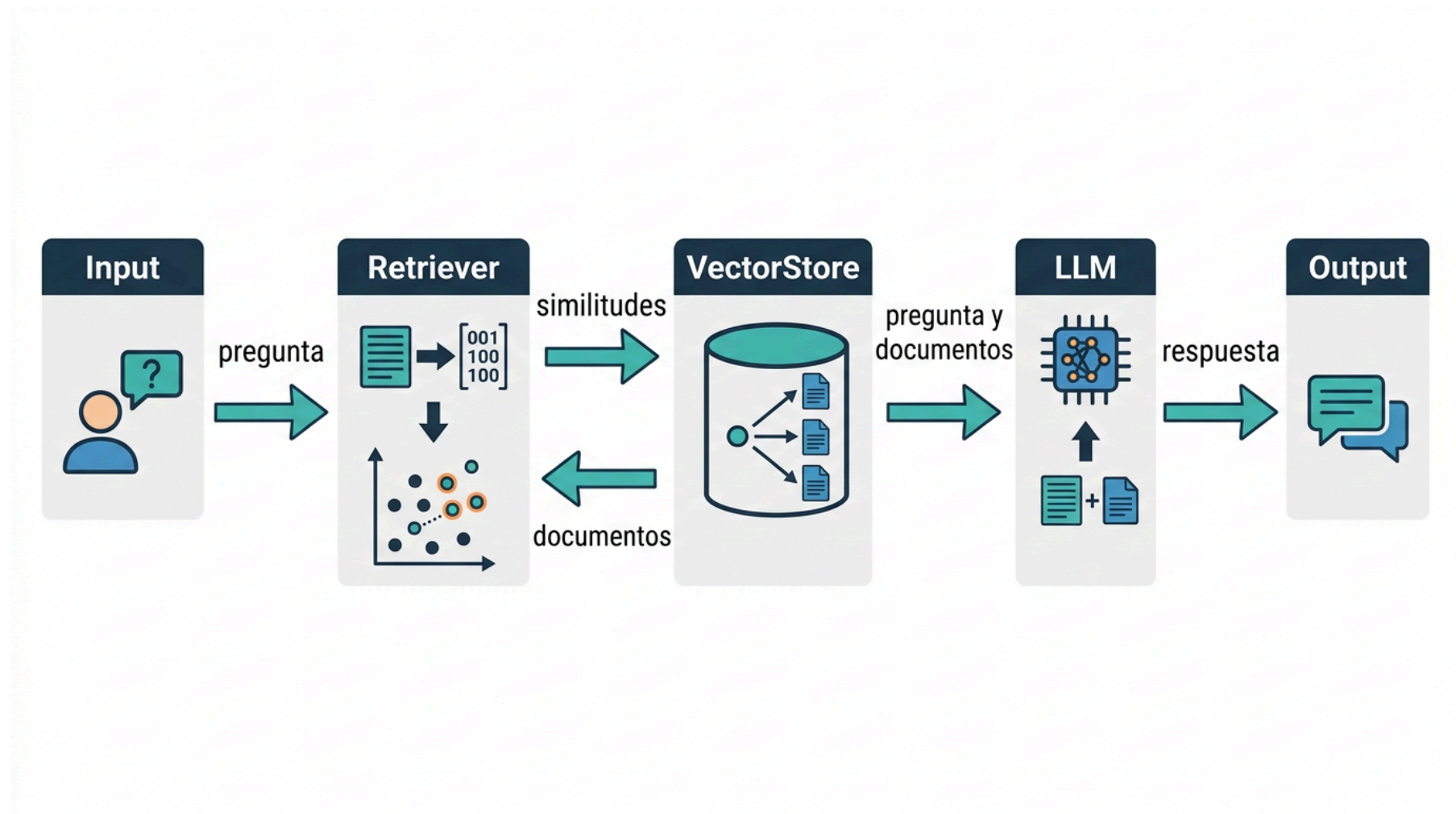
Construir un RAG es la parte fácil; saber si responde bien es el problema real. En esta charla implementamos desde cero, en Python puro, las métricas que usan frameworks como RAGAS y DeepEval: faithfulness, contextual precision y contextual recall. Primero el pseudocódigo y la intuición matemática, luego el mismo cálculo usando las librerías, para que quede claro qué hace cada una por debajo. Cerramos con las trampas prácticas: por qué un juez LLM da scores inconsistentes, cómo se calibran, y cuánto cuesta evaluar a volumen real.

Table of Contents

1. Intro
2. ¿Cómo funciona un LLM como Juez?
3. ¿Cómo puntúa un LLM como Juez?
4. Limitaciones y Riesgos de un LLM como Juez
5. Faithfulness RAGAS
6. Faithfulness DEEPEVAL
7. Implementación
8. Implementación DeepEval
9. Calibración

Intro

Retrieval Augmented Generation (RAG) es una arquitectura o patrón de diseño que permite conectar un modelo de lenguaje (LLM) con una fuente de datos externa para mejorar sus respuestas.

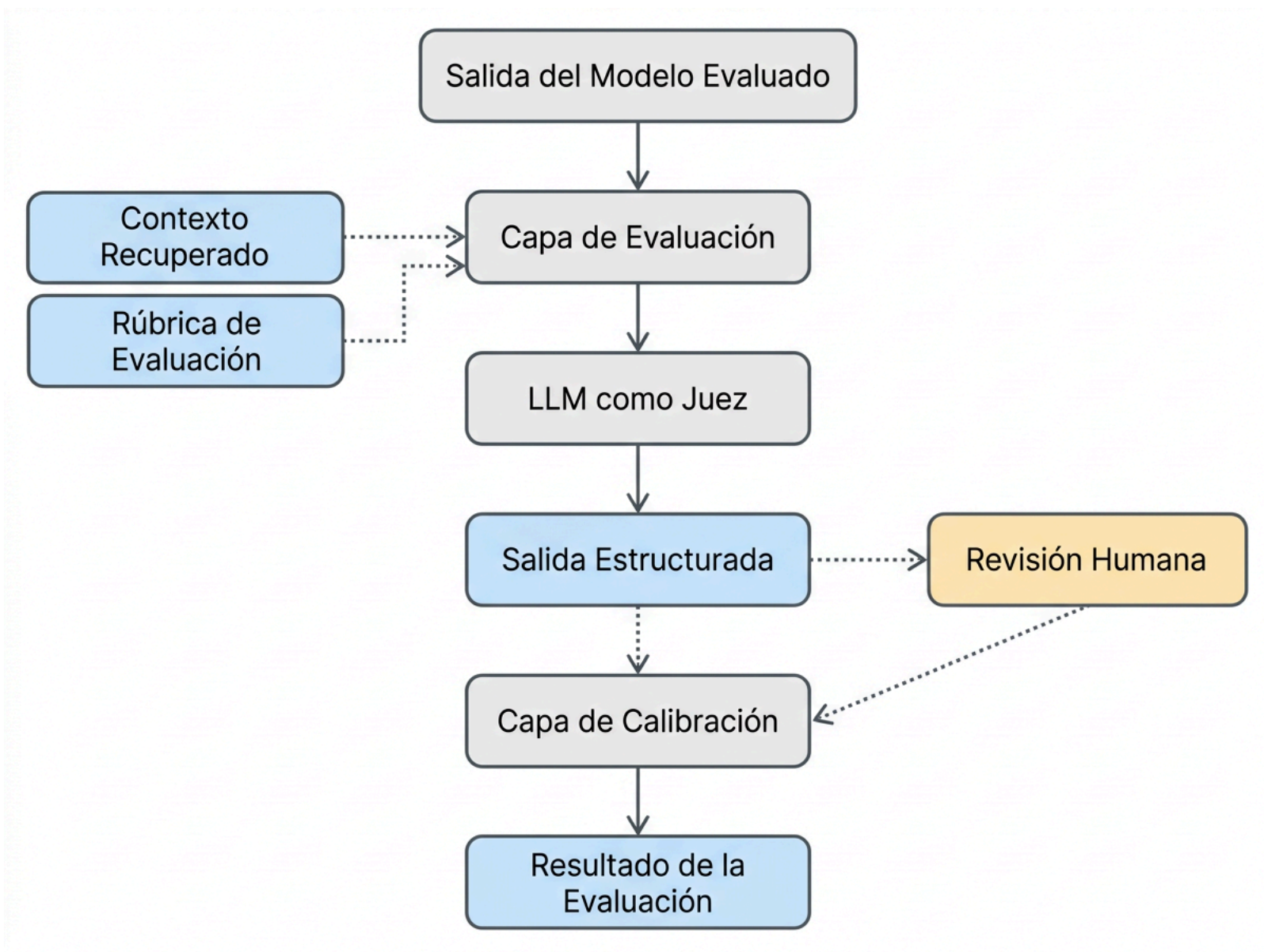


¿Cómo funciona un LLM-as-a-Judge?

A primera vista parece sencillo el funcionamiento de un LLM como Juez, un modelo evalúa la salida de otro modelo sin embargo hay que considerar ciertos aspectos críticos para determinar si el resultado que se produce es confiable y reproducible.

Para la evaluación de las respuestas de un modelo se refina el prompt con el que se evaluará la respuesta recuperando el contexto y una rúbrica de evaluación que se inyectarán en el prompt a enviar al juez y la salida debe ser apto para ser leído por la máquina es decir un formato json.

Para que la salida sea consistente se debe calibrar usando técnicas de aprendizaje automático.



¿Cómo puntúa un LLM-as-a-Judge?

Existen 3 tipos de tipos de puntuaciones.

1. Pairwise Prompt

El Juez se le presenta en el prompt dos respuestas y después se le pide que seleccione la mejor.

2. Pointwise Prompt

El Juez se le presenta en el prompt la pregunta y respuesta, después se le pide que califique usando una escala tipo 1-5.

3. Binarywise Prompt

El Juez se le presenta en el prompt un enunciado y se le pide que distinga entre verdadero o falso dependiendo de lo que se evalúe

Limitaciones y riesgos de un LLM como Juez

Utilizar un LLM como Juez puede traer beneficios, sin embargo, puede introducir puntos ciegos en el proceso de evaluación y entre las principales dificultades se encuentran:

Sesgo de posición

El juez puede dar preferencia a una propuesta que se encuentre al último o al principio del prompt dependiendo del modelo.

Sesgo de verbosidad

El juez puede dar preferencia a respuestas más largas o dar una puntuación a más alta a respuestas largas y prolijas en lugar de a una breve y clara.

Sesgo de auto-valorización

Es posible que un modelo de preferencia a respuestas redactadas por su propia familia de modelos, es decir, al modelo evaluador le suele gustarle la redacción y estructura que le resultan familiares.

Sensibilidad a las indicaciones

Hay que tener mucho cuidado porque un cambio en la rúbrica puede afectar a las puntuaciones finales.

Desviación del Modelo

Tomar en cuenta que los proveedores de modelos actualizan de forma constante y sin previo aviso. Cuando ocurre esto se puede desajustar las calificaciones de los modelos.

Optimización Adversarial

Al ajustar el modelo con el mismo evaluador en el entrenamiento puede resultar que el modelo aprenda los patrones que ofrecen mayor puntuación en lugar ofrecer mejores respuestas.

Faithfulness

Es una métrica que mide como la consistencia factual de una respuesta esta con el contexto recuperado, es decir, mide que tanto de la respuesta esta fundamentado en el contexto recuperado y si la respuesta empieza a desviarse del contexto la métrica penaliza la respuesta dandole un puntaje menor. El rango de la métrica es entre 0 y 1.

Sin embargo dependiendo de la librería que se utilice es como se evalúa la respuesta.

Faithfulness RAGAS

INPUT:

```
q    : question      (string)
a    : answer/output  (string)
c    : context = passages (list[string])
LLM  : judge model
```

OUTPUT:

```
F    : faithfulness score in [0, 1]
```

```
FUNCTION ragas_faithfulness(q, a, c, LLM):
```

```
# Step 1 – Statement extraction FROM THE ANSWER
```

```
S = LLM.extract_statements(q, a)
```

```
# Step 2 – Verify each statement AGAINST THE CONTEXT
```

```
supported = 0
```

```
FOR s_i IN S:
```

```
    # "can s_i be inferred from context?" -> Yes/No (+reason)
```

```
    verdict = LLM.verify(s_i, c)
```

```
    # <-- POSITIVE support required
```

```
    IF verdict == YES:
```

```
        supported += 1
```

```
# Step 3 – Score
```

```
F = supported / len(S)
```

```
RETURN F
```

Faithfulness DeepEval

```
INPUT:
    input          : user query (string)      # required param, NOT scored
    actual_output   : generated output (string)
    retrieval_context : passages (list[string])
    LLM             : judge model

OUTPUT:
    score : faithfulness score in [0, 1]

FUNCTION deepeval_faithfulness(actual_output, retrieval_context, LLM):

    # Step 1 – Extract TRUTHS from the CONTEXT
    truths = LLM.generate_truths(retrieval_context, limit)

    # Step 2 – Extract CLAIMS from the OUTPUT
    claims = LLM.generate_claims(actual_output)

    # Step 3 – Per-claim verdict: does it CONTRADICT the truths?
    verdicts = []                                # each verdict in {"yes", "no", "idk"}
    FOR claim IN claims:
        v = LLM.generate_verdict(claim, truths)
        # "no" ONLY if truths DIRECTLY contradict the claim
        # "idk" if not mentioned / unverifiable
        # "yes" if it agrees
        verdicts.append(v)

    # Step 4 – Score = fraction of claims NOT contradicted
    faithful = COUNT(v IN verdicts WHERE v != "no")    # <-- "yes" AND "idk" both pass
    score = faithful / len(verdicts)

    RETURN score
```

Implementación Faithfulness RAGAS

```
def faithfulness_ragas(prompt: str, response: str, context: str):
    claims = split_sentences(response)
    supported = 0
    for claim in claims:
        content = VERIFY_RAGAS.format(**{"claim": claim})
        result = call_openai(
            messages=[
                {"role": "system", "content": DEFAULT_SYSTEM_PROMPT},
                {
                    "role": "user",
                    "content": """Pregunta: {prompt}\n\nConexto: {context}\n\nContenido: {content}""".format(
                        **{"prompt": prompt, "context": context, "content": content}
                    ),
                },
            ],
            schema=RagasEntailment,
        )
        supported += int(result.entailed) # type: ignore
    return supported / len(claims)
```

```
faithfulness_ragas(  
    "Quien fue Nikola Tesla? y Cuáles fueron sus contribuciones?",  
    "Fue un físico del sigo XIX y contribuyo al diseño de la corriente alterna",  
    "Nikola Tesla fue un ingeniero, futurista e inventor serbio-estadounidense."  
    "Es conocido por sus contribuciones al diseño del sistema moderno de suministro eléctrico de corriente alterna. "  
    "Nacido y criado en el Imperio austrohúngaro, Tesla estudió ingeniería y física en la década de 1870, aunque no obtuvo ningún título.",  
)
```

```
entailed=True justification='El contexto menciona que Nikola Tesla fue un ingeniero y inventor conocido por sus contribuciones al suministro eléctrico de corriente alterna y que estudió ingeniería y física en la década de 1870, lo que permite inferir que fue un físico del siglo XIX y que efectivamente contribuyó al diseño de la corriente alterna.'
```

1.0

```
faithfulness_ragas(  
    "Que día es hoy?",  
    "Hoy es Lunes 13 de Agosto de 2026",  
    "La fecha de hoy es 11 de Agosto de 2026",  
)
```

```
entailed=False justification='El contexto indica que la fecha es 11 de Agosto de 2026,  
por lo tanto, no puede afirmarse que hoy sea Lunes 13 de Agosto de 2026.'
```

```
0.0
```

Implementación Faithfulness DeepEval

```
def faithfulness_deepeval(prompt: str, response: str, context: str):
    truths = call_openai(
        messages=[
            {"role": "system", "content": DEFAULT_SYSTEM_PROMPT},
            {
                "role": "user",
                "content": GENERATE_TRUTHS.format(**{"context": context}),
            },
        ],
        schema=Truths,
    )
    claims = split_sentences(response)
    verdicts = []
    for claim in claims:
        verdict = call_openai(
            messages=[
                {"role": "system", "content": DEFAULT_SYSTEM_PROMPT},
                {
                    "role": "user",
                    "content": VERIFY_DEEPEVAL.format(
                        **{"truths": truths.truths, "claim": claim} # type: ignore
                    ),
                },
            ],
            schema=ScoreResponse,
        )
        verdicts.append(verdict)
    return sum([v.score for v in verdicts]) / len(verdicts)
```

```
faithfulness_deepeval(  
    "Quien fue Nikola Tesla? y Cuáles fueron sus contribuciones?",  
    "Fue un físico del siglo XIX y contribuyó al diseño de la corriente alterna",  
    "Nikola Tesla fue un ingeniero, futurista e inventor serbio-estadounidense. "  
    "Es conocido por sus contribuciones al diseño del sistema moderno de suministro eléctrico de corriente alterna. "  
    "Nacido y criado en el Imperio austrohúngaro, Tesla estudió ingeniería y física en la década de 1870, "  
    "aunque no obtuvo ningún título.",  
)
```

```
[ScoreResponse(score=1.0, justification='Las verdades presentadas son consistentes con la afirmación.  
Nikola Tesla fue un ingeniero e inventor del siglo XIX y su contribución al  
diseño del sistema de corriente alterna es un hecho conocido.  
No hay ninguna verdad que contradiga directamente la afirmación.')]
```

1.0

```
faithfulness_deepeval(  
    "Que día es hoy?",  
    "Hoy es Lunes 13 de Agosto de 2026",  
    "La fecha de hoy es 11 de Agosto de 2026",  
)
```

```
[ScoreResponse(score=0.0, justification='La afirmación de que hoy es Lunes 13 de Agosto de 2026 contradice  
la verdad de que la fecha de hoy es 11 de Agosto de 2026, ya que no puede ser ambas fechas al mismo tiempo.')]
```

```
0.0
```

Implementación DeepEval

```
@observe(
    metrics=[AnswerRelevancyMetric(verbose_mode=True),FaithfulnessMetric(verbose_mode=True)]
)
def answer_question(state):
    question = state["question"]
    documents = retriever.invoke(question)
    answer = main_agent.invoke(
        {"messages": [{"role": "user", "content": question}]},
        context={"documents": documents}, # type: ignore
    )
    update_current_span(
        test_case=LLMTestCase(
            input=question,
            actual_output=answer["messages"][-1].content,
            retrieval_context=[d.page_content for d in documents],
        )
    )
    return {
        "question": question,
        "answer": answer["messages"][-1].content,
    }
```

```
dataset = EvaluationDataset(  
    goldens=[  
        Golden(  
            input="Que problema se concentra en los tickets de Initech?",  
            multimodal=False,  
        ),  
        Golden(  
            input="Porque la cuenta Acme declino en el Q2 de este año?",  
            multimodal=False,  
        ),  
    ]  
)  
for golden in dataset.evals_iterator():  
    app.invoke(  
        {  
            "question": golden.input,  
            "answer": "",  
        },  
        config={"callbacks": [CallbackHandler()]},  
    )
```

Calibración

El tema de calibración es importante porque las respuestas de un LLM pueden sufrir de sesgos que se mitigan con las estrategias: intercambio de orden, normalización de longitud, ensembles de familias distintas y forzando el razonamiento antes del veredicto.

Calibración por Sondeo

Este tipo de técnica suele abordar el problema de una forma diferente puesto que de la capas intermedias se extraen las activaciones y se ajusta un modelo clasificador para predecir un si la respuesta será correcta o incorrecta.

¿Por qué en capas intermedias?

Los Modelos en sus capas iniciales suelen ser de bajo nivel, es decir carecen de una representación rica. Las capas más profundas están muy especializadas para la predicción del siguiente token, en contraste las capas intermedias retienen la mayor representación semántica de toda la red.

Métricas

Para evaluar la precisión del modelo se usan métricas como Kuiper y Expected Calibration Error las cuales miden el desajuste acumulado de calibración y la cuantificación entre la confianza predicha y la precisión real respectivamente.

Muchas Gracias!

Github: github.com/jmanuelc87